



Browser automation, recorded once.

Record a browser task once. Run it again with new inputs, forever.

VERSION	STATUS	CURRENCY	PLATFORM
2.0	Shipped	BDT (Tk)	Chrome MV3 + Web

Contents

01	What the automator does
02	How it works
03	Field recognition
04	Features
05	Pricing
06	API
07	Not in this release

What the automator does

Anything you do in a browser more than once,
you should only have to do once.

Apify watches you perform a task in your own browser — booking a room, searching a listing, filling a form — and learns it. Not a screen recording: it captures every step, works out which inputs are the ones that change between runs, and turns those into a form.

From then on, you fill in that form instead of doing the task. Apify replays the whole thing on a headless browser on the server, scrapes the results, and hands them back as structured data. You never touch that form again.

Every automation also gets its own API key, so anything you record becomes an endpoint you can call from your own code.

How it works

01 Record

Click record in the extension and do the task normally. Apify captures each click and input across every site you touch, classifying fields as it goes. The popup shows a live count of steps, fields, sites, and URL parameters.

02 Derive

Stop recording and it hands off straight to your dashboard. Apify works out which recorded inputs are variable, names them in plain language, and prices the run. Optionally it scans the page again for fields the recording missed.

03 Edit

Real sites leak junk into the form — opaque URL parameters, internal IDs. Remove what you don't want to see, turn on/off filters into checkboxes, or just describe the change you want in plain English.

04 Run

Fill the form with new values and run it. A headless browser replays the steps, handles popups, scrapes the results, and returns clean structured data — from the dashboard or over the API.

Field recognition

A recorded input is only useful if Apify knows what kind of control it is. Each captured field is classified so the replay drives it correctly — a calendar is not typed into, a stepper is not clicked once.

TYPE	WHAT IT COVERS
text	Free-text inputs and text areas.
dropdown	Native selects and select-like widgets.
calendar	Date pickers, including non-input widgets that open a calendar.
tickmark	Checkboxes and toggles.
stepper	Plus/minus counters, including unlabelled booking-style steppers.
nested	Fields that only appear after another field is used — a guest stepper revealing a per-child age dropdown.

To relocate a field on replay, Apify tries an ordered fallback chain — label text, position, DOM structure — and only calls in an AI page scan when every signal in that chain has failed. It is not run on every replay.

Features

Recorder

Cross-site capture	One recording can span more than one website.
Live counters	Steps, fields, sites, and URL parameters update as you record.
Direct handoff	Stopping opens your dashboard with the recording already loaded.
Copy or export	Take the recording as raw JSON if you'd rather handle it yourself.

Field editor

Remove a field	Hides it from the form but keeps its recorded value at replay, so search-critical parameters keep working.
Detect filters	Finds which fields are really on/off filters and turns them into checkboxes. Unticking one drops it from the request entirely.

Plain-English edits	Describe the change — "make the 5-star field a checkbox" — and it's applied, or refused with a reason.
Scan for missing fields	Checks the live page for inputs the recording didn't catch.
Reversible	Edits are an overlay on the recording. Restore a removed field at any time; the original is never altered.
Runs	
Headless replay	Runs server-side on a real browser — your machine stays free.
Popup handling	Cookie banners and interstitials are dismissed automatically.
Structured results	Scraped result cards are normalized and returned as clean records, not raw HTML.
Output count	Choose how many results to pull; the run price scales with it.
Live progress	Watch a run's stage as it happens rather than waiting blind.
Fixing a bad run	
Report	Say what came out wrong in your own words. Apify re-extracts from the raw capture and shows a corrected preview — nothing is saved yet.
Confirm	If the preview looks right, the correction is kept as a standing note and applied to every future run of that automation.
Account and API	
Email sign-in	Email and password, nothing else. Your session survives a reload.
Recording is safe	A recording made while signed out is held and restored once you sign in — you never lose it to the login.
Per-automation key	Every automation carries its own API key, passed as <code>X-API-Key</code> .
Wallet balance	Balance and current plan are visible from the account menu.

05

Pricing

Plans are billed monthly in BDT and can be cancelled at any time. Billing runs through dCash, with card and bank transfer also available.

TIER	MONTHLY	INCLUDED
Free	Tk 0	5 automation builds per day. Good for trying things out.
Builder	Tk 1,500	30 automation builds per day. Unlimited runs on everything you've saved. Priority support.
Pro	Tk 3,500	Unlimited builds and unlimited runs. Marketplace seller access at lower fees. Early access to new features.
Enterprise	Custom	Everything in Pro, plus team accounts, dedicated support, and custom billing arrangements.

Per-run rate

Each automation is quoted a per-run rate when it's created. The rate is deterministic — it comes from how many steps the recording has and how many results you ask for, never from a guess.

QUOTED RATE — BDT

$$\text{cost} = 1.5 + (0.40 \times \text{steps}) + (0.20 \times \text{results})$$

$$\text{price} = \max(10, \text{roundUpTo5}(\text{cost} \times 4))$$

A 12-step automation pulling 10 results costs Tk 8.30 in compute and is quoted at Tk 35. Push it to 30 steps and 50 results and the same automation is quoted at Tk 95. Rates never fall below Tk 10 and always land on a round number.

06

API

Every endpoint below is scoped to a single automation and authenticated with that automation's key, sent as `X-Api-Key` or a bearer token.

ENDPOINT	DOES
<code>POST</code> /api/automations	Create an automation from a recording.
<code>GET</code> /api/automations	List your automations.
<code>GET</code> /api/automations/:id	Fetch one, with its input form.
<code>GET</code> /api/automations/:id/quote	Price a run before committing to it.
<code>POST</code> /api/automations/:id/run	Run it with new values, return results.
<code>GET</code> /api/automations/:id/progress	Stage of the run in flight.
<code>GET</code> /api/automations/:id/runs/:runId	Fetch a past run and its output.

ENDPOINT	DOES
POST /api/.../runs/:runId/report	Preview a corrected extraction.
POST /api/automations/:id/report-confirm	Keep the correction for all future runs.
PATCH /api/automations/:id/fields	Hide, restore, or relabel a field.
POST /api/.../fields/detect-filters	Turn on/off filters into checkboxes.
POST /api/.../fields/command	Apply a plain-English field change.
POST /api/.../fields/scan	Look for fields the recording missed.
DELETE /api/automations/:id	Delete an automation.

07

Not in this release

Named here so the boundary is explicit — these are designed but not yet shipped.

- **Marketplace.** Reselling your automations to other users per run, with a 70/30 margin split, buyer-scoped keys, and seller listings. Advertised on the Pro tier; the storefront itself is still to come.
- **Saved logins.** Per-site sessions so a run can stay signed in to the site it's automating.
- **Data piling.** Feeding the output of one site into an input on the next, within a single multi-site automation.
- **Team accounts.** Shared ownership of automations under one organization.

Open item. The internal spec defines plan limits as monthly run pools (Free — 150 runs per month, 10 per day). The published pricing page sells them as daily build limits. This document follows the published page. The two need reconciling before the next billing change.